OptiPhase-Σ: Measured Pass Interference and Bounded Refinement Checks in the LLVM New Pass Manager

Devarsh Patel[1], Darshil Vaid[1] and Kanagaraj R[1*]

[1*]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

*Corresponding author(s). E-mail(s): kanagaraj.r@vit.ac.in;
Contributing authors: devarsh.patel2024@vitstudent.ac.in;
darshil.vaid2024@vitstudent.ac.in;

Abstract

Optimization pipelines in LLVM are sequences of passes whose effects depend on order and on each other. We ran 28,800 pipeline variants over 60 kernels from four benchmark suites and measured the interactions between pairs of 15 passes. The main measure is the exact instruction count of the optimized code. Of the 225 ordered pairs, 116 have a mean interaction that exceeds two standard errors across kernels. Where a second application of a pass leaves the code unchanged, the self-interaction equals minus the pass's single effect, and this identity holds in all 331 such cases. Wall-clock timing resolves few pass interactions: its median noise is 0.73% of the canonical cost, and the 26 of 225 timing interactions that pass a two-standard-error test lie below the noise floor. Redundant pass executions are common, with a mean ratio of 0.633. A block-local checker encodes 26.7% of matched block pairs. Within the encoded names, 544,249 are proved to refine, and 13,523 are violated and are candidates for inspection. Of the Pareto-front members, 45.6% are proved to refine, 4.4% have candidate counterexamples, and the rest are not comparable, so the study does not establish that Pareto-optimal schedules refine their canonical builds. The results are bounded by one compiler release, one timing lowering level, and one input per kernel. Of the 28,800 sweeps, 28,796 reproduce the canonical output under the -O0 lowering used for timing. For the 4 sweeps that differ, the optimized IR reproduces the canonical output under interpretation and -O2 lowering.

Keywords: phase ordering, LLVM new pass manager, pass interference, translation validation, Z3, hypervolume

1 Introduction

Compilers apply a sequence of optimization passes, not a single optimization, and the passes depend on one another. One pass can create the pattern that a later pass needs, remove information that another pass relies on, or change a cost that a later decision reads. This is the phase-ordering problem. Its size on real code is seldom measured directly. Most published experiments report which sequence performs best, not how much each pair of passes changes the result.

This paper measures those interactions in the new pass manager of LLVM [1]. We ran 28,800 pipeline variants over 60 kernels from four benchmark suites. The main measure is the instruction count of the optimized IR, which is exact and independent of timing and of the backend. We also timed every variant against the canonical build of its kernel in one measurement window, and we checked every variant's optimized IR against its canonical IR. The results are stored in a telemetry table with 26 fields per sweep. We ask four questions.

1. How much does the order of two passes change the code, and which pairs interact most? (Section 5.2.)
2. How much of that change is visible in the execution time, given the noise of the timing measurements? (Section 5.4.)
3. How many pass executions in a standard pipeline leave the IR unchanged, and what time do those executions consume? (Section 5.6.)
4. How far can a bounded, machine-checked comparison of optimized code go towards showing that each transformation preserves meaning? (Section 5.8.)

Our contributions are the following.

- An empirical campaign of 28,800 sweeps. It covers the single passes, every ordered pair of 15 passes, a stepwise ablation of the `-O3` pipeline, an interference study around `loop-vectorize`, and sequences chosen from the instruction-count tensor.
- An instruction-count interaction, normalized by the canonical count, and an identity for self-interactions that holds exactly in all 331 kernel-pass cases where it applies.
- IR-level checks of every pass on two canonical forms, which show that the canonical form changes the effect of `loop-unroll`.
- A timing measure with its noise floor, and the finding that timing resolves few pass interactions, all of them below the noise floor.
- A block-local refinement checker for integer LLVM IR. We state the soundness argument for its UNSAT answers, we check its encoding exhaustively against an independent reference semantics for small widths, and we report where its answers stop being evidence of correctness.

2 Background and Related Work

This section discusses recent work from the compiler community that is closest to the study. Each cited paper is described by its contribution, and the comparison with the present study is stated at the end of each subsection. Tools and benchmarks are cited where the methods use them.

2.1 Phase ordering and pass interactions

Choosing passes and their order is a search problem. Ashouri et al. present the Protean Compiler, which orders optimizations at fine granularity under the guidance of a learned predictor [2]. Han and Kim expand the phase-ordering search space with dormant information, and they prune it more conservatively than methods that group transformations by expert knowledge [3]. Xie et al. introduce optimization interaction, in which several phases, among them inlining and loop unrolling, act on the same code, and they fuzz a JVM compiler to maximize such interactions [4]. Seeker et al. discover heuristics that are embedded in a compiler rather than exposed as flags, and they tune those heuristics automatically [5]. Lenfers et al. model schedules as graph-structured objects, so that one auto-scheduling framework can serve several compiler infrastructures [6]. VenkataKeerthy et al. connect model development in Python to an optimizing compiler, so that learned models can make decisions inside it [7]. Cummins et al. train language models on LLVM IR and assembly and apply them to optimization tasks [8]. Gassmann et al. evaluate LLVM optimizations for zero-knowledge virtual machines, whose costs differ from those of CPUs, and they show that a few zkVM-aware changes to LLVM passes help [9]. dos Santos et al. show that optimization flags change the error rate of GPU programs, so that the effects of optimization reach beyond speed [10].

Much of the work above searches over pass sequences or measures the effect of a single optimization configuration. The present study instead measures how strongly the effect of one pass depends on the passes that run before it, in instruction count, and how much of that dependence the timing data can resolve.

2.2 Vectorization, loop transformations, code size, and compile time

Alladi et al. enable compiler-driven vectorization of Transformer models for edge devices, including RISC-V systems. Where the existing auto-vectorizer fails, they add a reduction-detection pass and a transpose step [11]. Murtovi et al. pair control-flow unmerging with loop unrolling on GPUs, so that the combination unlocks follow-on optimizations that neither technique reaches alone [12]. Zhi et al. retrieve demonstrations for a language model that rewrites loops in static control parts, and they check each candidate for equivalence with mutation, coverage, and differential testing [13]. H. Kwon et al. propose a chain of verification for language-model-generated transformations. Cheap static screens run first, symbolic equivalence checks based on Alive2 [14] run next, and run-time checks cover what cannot be proved statically. The chain widens vectorization coverage in LLVM [15]. Italiano and Cummins use a language model together with differential testing to find missed code-size optimizations in C and C++ compilers [16]. Yuan et al. shrink linked binaries by outlining repeated instruction sequences, and they use runtime profiles to balance size against speed [17]. Saito et al. merge three or more similar functions at once, using multiple sequence alignment, where earlier approaches merged only pairs [18]. Cugurovic et al. classify methods as cold or warm with a learned model, and they send cold methods to size-reducing transformations and warm methods to speed-oriented ones [19].

Mu et al. use sensor specifications to guide LLVM optimizations [20]. Schwarz et al. describe TPDE, a back end that analyzes the code once and then generates it in a single sweep [21]. Engelke and Schwarz compare how quickly several code generators compile query code [22]. Titzer analyzes single-pass WebAssembly compilers and their trade-offs [23]. Han et al. make the compiler stateful, so that incremental builds skip passes that were dormant in earlier builds [24].

These studies report compile time, code size, or run time for their chosen configurations. The present study instead measures how a pair of passes interacts in instruction count, and it uses timing as a second, noisier measurement of the same interaction.

2.3 Verification, miscompilations, and testing

Fehr et al. write MLIR verification dialects that state what operations mean, so that optimizers and verifiers can rely on a shared semantics, and they report defects that they found in upstream MLIR [25]. Xu et al. couple a language model with a formal verifier in a feedback loop to find peephole optimizations that LLVM misses [26]. Popescu and Lopes measure how much performance undefined-behavior-based optimizations deliver across processor families, and they find only slight end-to-end gains in LLVM [27]. Mao et al. apply custom LLVM passes and bytecode rewriting to eBPF programs before the kernel verifier runs, and they confirm that the verifier accepts the tuned programs [28]. Fan and Regehr integrate mutation-based fuzzing with Alive2 inside the LLVM middle end, so that mutation, optimization, and verification run in one process [29]. J. Kwon et al. combine optimization-directed grey-box fuzzing with translation validation, to find incorrect optimizations in compilers that change continuously [30]. Lobo et al. set out a roadmap to remove undef from LLVM IR, because the interaction of undef with ordinary rewrites makes simple algebraic transformations unsound, and the semantics of undefined and poison values are subtle [31, 32]. Yu et al. present Ratte, which uses composable semantics for MLIR dialects to build modular fuzzers that find miscompilations [33].

Most of the work above targets individual optimizations or the compiler as a whole. The checker used here runs on every sweep, one module at a time, and it covers integer code only, as Section 4 states.

3 Formal Framework

3.1 Schedules, costs, and interactions

Let G be a kernel in canonical form, the module after `mem2reg`. A stage π is one pass invocation, and a schedule $\sigma = (\pi_1, \ldots, \pi_k)$ applies its stages in order to give $\sigma(G)$. We write $\pi_j \circ \pi_i$ for the schedule that applies π_i and then π_j.

Let J be a cost of a module. The cost change of a schedule is

$$\Delta_{\sigma}^{J}(G) = J(\sigma(G)) - J(G). \tag{1}$$

For two stages, the interaction is the part of the pair's cost change that its two single changes do not explain,

$$\begin{aligned} I_{ij}^{J}(G) &= \Delta_{ij}^{J}(G) - \Delta_{i}^{J}(G) - \Delta_{j}^{J}(G) \\ &= \Delta^{J}(\pi_j \mid \pi_i(G)) - \Delta^{J}(\pi_j \mid G), \end{aligned} \tag{2}$$

where $\Delta^{J}(\pi_j \mid \pi_i(G)) = J(\pi_j(\pi_i(G))) - J(\pi_i(G))$ is the change that π_j causes after π_i. The interaction is zero when the two passes act independently on G for the cost J. A positive value means that the pair's combined change is larger than the sum of its single changes.

The study uses two costs. The instruction count N of the optimized module is an exact integer, measured before lowering, so the backend does not affect it. The wall-clock time J is measured with noise (Section 4.5).

3.2 Instruction-count interaction

We normalize the instruction-count interaction by the canonical count $N(G)$,

$$\tilde{I}_{ij}(G) = \frac{I_{ij}^{N}(G)}{N(G)}, \tag{3}$$

and average it over the K kernels to give the tensor $\mathbf{M}_{\text{pass}}^{N}$. Its entries are fractions of the canonical instruction count.

Remark 1 (Self-interaction identity) Suppose that a second application of π_i leaves the module unchanged. Then $\Delta_{ii}^{N}(G) = \Delta_{i}^{N}(G)$, and the self-interaction is $\tilde{I}_{ii}(G) = (\Delta_{ii}^{N}(G) - 2\Delta_{i}^{N}(G))/N(G) = -\Delta_{i}^{N}(G)/N(G)$. The identity is exact, because the counts are integers of the same module. Section 5 checks it on every kernel where the premise holds.

3.3 Timing interaction

The timing interaction is divided by the canonical time of the sweep's window,

$$\tilde{I}_{ij}^{t}(G) = \frac{I_{ij}^{t}(G)}{J_0(G)}, \tag{4}$$

where I_{ij}^{t} is the interaction of the wall-clock cost. Both deltas of a pair are measured in one exclusive window, and the noise is estimated from the sweeps whose optimized IR equals the canonical IR (Section 4.5).

3.4 The single-effect ratio baseline

This baseline defines the transition tensor with a floor in the denominator,

$$[\mathbf{M}_{\text{pass}}^{\text{ratio}}]_{ij} = \frac{1}{K} \sum_{G} \frac{I_{ij}^{t}(G)}{\max\big(\varepsilon,\ |\Delta_{j}^{t}(G)| + |\Delta_{i}^{t}(G)|\big)}, \tag{5}$$

with $\varepsilon = 1$ ns. Its denominator is built from two noisy single-pass measurements. It is unstable when both passes change the cost by about the noise level, and the floor only prevents division by zero. The paper reports this estimator as a diagnostic and does not use it for its conclusions.

3.5 Commutators

The commutator of two stages on one kernel is

$$\begin{aligned} [\pi_i, \pi_j]_J(G) &= \frac{J((\pi_j \circ \pi_i)(G)) - J((\pi_i \circ \pi_j)(G))}{J(G)} \\ &= \frac{\Delta_{ij}^{t}(G) - \Delta_{ji}^{t}(G)}{J(G)}. \end{aligned} \tag{6}$$

It is zero when the two orders cannot be told apart at the resolution of the measurement.

3.6 Redundant executions and compile-time waste

The pass manager runs each pass as a separate execution. An execution is redundant when the IR printed after it is unchanged from the IR before it. For a schedule σ, let $E(\sigma)$ be the number of pass executions and $R(\sigma)$ the number of redundant ones. The redundant-pass ratio is

$$\rho_{\text{RPB}}(\sigma) = \frac{R(\sigma)}{E(\sigma)}. \tag{7}$$

Let $P(\sigma)$ be the total pass time of the sweep's optimizer run, as its `-time-passes` report gives it. The pass time that redundant executions consume is estimated as

$$\mathcal{W}(\sigma) = P(\sigma)\, \rho_{\text{RPB}}(\sigma). \tag{8}$$

This estimate assumes that every execution costs the same share of $P(\sigma)$. The telemetry does not record the time of each execution, so the estimate is not a measurement of those times. The wall time of the optimizer process is not used, because it also includes process start-up, parsing, and trace output.

3.7 Objectives and the hypervolume

Each sweep is described by three objectives, all to be maximized. Let J_0 be the canonical cost measured in the sweep's own window, T_0 the canonical text size, and C_0 the

Table 1 Poison conditions of the modeled flags, on defined operands. Each condition is an exact predicate on the bit-vectors, and any instruction in the fragment whose condition holds yields poison.

Instruction	Flag	Poison when
`add`, `sub`, `mul`	`nsw`	the exact result lies outside the signed range of the width
`add`, `sub`, `mul`	`nuw`	the exact result lies outside the unsigned range of the width
`shl`	`nsw`	the signed value times 2^{amount} lies outside the signed range
`shl`	`nuw`	the unsigned value times 2^{amount} lies outside the unsigned range
`lshr`, `ashr`	`exact`	any bit shifted out is one
`or`	`disjoint`	the two operands share a set bit
`zext`	`nneg`	the operand is negative as a signed value
`trunc`	`nsw`, `nuw`	the truncated value, extended back, differs from the operand
`icmp`	`samesign`	the operands have different sign bits
`shl`, `lshr`, `ashr`	none	the shift amount is at least the bit width

compile time of the kernel's −O0 sweep. For a sweep with cost J, text size T, and compile time C, the objectives are

$$s = \frac{J_0}{J_0 + J}, \qquad b = \frac{T_0}{T_0 + T}, \qquad c = \frac{C_0}{C_0 + C}, \tag{9}$$

where C is the optimizer wall time recorded for the sweep. Each objective is $\frac{1}{2}$ when the sweep matches its reference, so the reference point of the hypervolume is $\mathbf{r} = (\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$. A schedule contributes only when it is better than the reference on all three objectives. The hypervolume of a finite set P is the Lebesgue measure of $\bigcup_{p \in P} [\mathbf{r}, p]$.

3.8 What an UNSAT answer establishes

The checker encodes a fragment $\mathcal{F}$ of LLVM integer code: the operations `add`, `sub`, `mul`, `and`, `or`, `xor`, `shl`, `lshr`, `ashr`, `icmp`, `select`, `zext`, `sext`, `trunc`, and `ret`, on values of at most 64 bits. Each instruction in $\mathcal{F}$ may carry only the flags of Table 1. A block containing any other instruction, operand, or flag is not encodable, and the checker reports it as such. Each defined value v is encoded as a pair $(\hat{v}, \bar{v})$ of a bit-vector and a poison flag. A live-in is a value used but not defined in the block.

For a name x defined in both blocks, the refinement query is

$$V_x = \neg \bar{x}_s \ \wedge \ \left(\bar{x}_t \ \vee \ \hat{x}_s \neq \hat{x}_t \right), \tag{10}$$

where the subscripts s and t denote the source and target blocks. A violation is an assignment of non-poison live-in values for which the source defines x without poison and the target does not define it equally.

Lemma 1 (Encoding agreement) *For each instruction in $\mathcal{F}$, if every operand's encoded pair agrees with the LLVM value and poison of that operand, then the instruction's encoded pair agrees with the LLVM result. The value term applies the operation to the bit-vectors. The poison*

Table 2 Exhaustive agreement between the Z3 encoder and the reference semantics, widths 1 to 4, per instruction family.

Family	Cases	Mismatches
add	404	0
add nsw	404	0
add nsw nuw	404	0
add nuw	404	0
and	404	0
ashr	404	0
ashr exact	404	0
icmp	4,040	0
icmp samesign	4,040	0
lshr	404	0
lshr exact	404	0
mul	404	0
mul nsw	404	0
mul nsw nuw	404	0
mul nuw	404	0
or	404	0
or disjoint	404	0
select	1,212	0
sext	28	0
shl	404	0
shl nsw	404	0
shl nsw nuw	404	0
shl nuw	404	0
sub	404	0
sub nsw	404	0
sub nsw nuw	404	0
sub nuw	404	0
trunc	74	0
trunc nsw	74	0
trunc nsw nuw	74	0
trunc nuw	74	0
xor	404	0
zext	28	0
zext nneg	28	0

term is the disjunction of the operands' poison bits, the shift-amount condition, and the flag conditions of Table 1. For `select`*, the poison term depends only on the chosen arm and the condition.*

The lemma has two supports. The first is by construction. Each flag condition is an exact predicate on the bit-vectors. The overflow predicates widen the operands by one bit for `add` and `sub`, and by the full width for `mul`. The second is by exhaustive evaluation. The production encoder is run on every concrete case for widths up to 4, with every defined value and one poisoned operand, and each result is compared with an independent reference written from the poison rules. The check covers 19,368 cases in 34 instruction families, with 0 disagreements (Table 2). The exhaustive check covers small widths only, so the lemma for larger widths rests on the construction argument.

Soundness of the query. Let $B_s, B_t \in \mathcal{F}$ have the same live-in names and widths, and let x be a name defined in both blocks. Suppose that no assignment of non-poison values to the live-ins satisfies V_x. Take an assignment under which B_s defines x without poison. By Lemma 1, applied to the instructions of each block in definition order, the encoded pairs of every name agree with LLVM under that assignment. So $\bar{x}_s$ is false, and $\hat{x}_s$ is the value of x in B_s. If B_t did not define x without poison, or computed a different value, then $\bar{x}_t \vee \hat{x}_s \neq \hat{x}_t$ would hold, and the assignment would satisfy V_x. That contradicts the assumption. So B_t defines x without poison and computes the same value. This argument is the whole soundness claim, and it holds only as far as the lemma does.

The argument makes no claim about values defined in only one block, about live-ins that differ, about poison live-ins, or about facts that hold only in the surrounding function. The last limit matters in practice. An optimizer may add a `nuw` flag because a range fact from another block rules out wrap-around. A block-local check does not see that fact. It then reports a counterexample that the whole function never reaches. We therefore report a SAT answer as a candidate counterexample, and we separate it from behavior observed by execution.

3.9 Exact hypervolume

Exactness of the algorithm. The hypervolume is computed with exact rational arithmetic over the stored binary64 objective values. The routine `hypervolume_3d` first discards the points that are not strictly greater than the reference point in every coordinate. A discarded point has a box of zero measure, so discarding it does not change the union. The three-dimensional union is then cut into slabs along the third coordinate, and the cross-section of each slab is measured with a two-dimensional routine. That routine sorts the points by their first coordinate and adds one strip for each point that raises the running maximum of the second coordinate. The volume is the sum of the cross-sections times the slab thicknesses. The routine is covered by unit tests that accompany the artifact. The result is exact for the stored objective values, not for the measurements behind them, because each objective value is a ratio of measured times.

4 Experimental Methodology

4.1 Benchmarks and canonical IR

The benchmark set has 60 kernels. 30 come from PolyBench/C version `4.2.1` (beta) [34], obtained from its SourceForge mirror, with the archive’s `SHA-256` recorded in the manifest. 20 come from MiBench [35], whose per-category archives are published by the University of Michigan. A MiBench kernel is one invocation in the small-dataset script of each program in the automotive, network, security, and telecomm categories. The `security/pgp` program ships only a script that depends on a configured keyring, so it is excluded. 10 come from the TSVC loops [36], which are taken from the LLVM test suite, and from the standalone kernels of CoreMark [37]. Evaluations of vectorizing compilers appear in the literature [38].

Each kernel is compiled to unoptimized bitcode with `clang -O0 -Xclang -disable-O0-optnone -emit-llvm -c`, linked with its own libraries where needed, and canonicalized with `opt -passes=mem2reg,loop-simplify,lcssa`. The sweeps use this canonical form. Section 5.3 measures how the `mem2reg`-only form, which omits the `loop-simplify` and `lcssa` steps, changes the passes. A canonical module passes three checks. It must pass `llvm-dis` validation. Two native runs must produce the same output digest. Interpreting the canonical bitcode with `lli` must give the same digest. All 60 kernels passed.

Two kernels read uninitialized automatic variables under plain `-O0` builds, so their native runs disagreed. Zero-initializing automatic variables with `-ftrivial-auto-var-init=zero` removed the disagreement, and the flag is applied to every build of those kernels. Two programs print elapsed time. Those fields are masked before digests are compared, and nothing else is masked.

4.2 The sweep campaign

Each kernel is swept under six regimes, which give 480 sweeps per kernel and 28,800 in all.

R1 The standard levels `default<O0>` to `default<O3>`, and the size levels. LLVM 23.1.3 rejects the names `default<Os>` and `default<Oz>`, so each size level is `default<O2>` with the `optsize` or `minsize` attribute forced on every defined function. This gives 360 sweeps.
R2 Each single pass of the pass set. 76 of the 84 canonical passes are valid in this LLVM. The other 8 are replaced by 8 substitute passes, chosen by a rule that was fixed before any result was seen. The rule and the whitelist are in the configuration directory. The pass `unify-loop-exits` crashes `opt` on two kernels, as described below, so it is replaced by `newgvn` in its own R2 slot on every kernel before any sweep runs. This gives 5,040 sweeps.
R3 Every ordered pair of 15 passes, including repeated passes, which gives 225 pairs. This gives 13,500 sweeps.
R4 Prefixes of the `default<O3>` pipeline, cut at the stages of LLVM's own nested text. There are 65 prefixes, which gives 3,900 sweeps.
R5 Interference around `loop-vectorize`. The 8 pre-pivot and 5 post-pivot cleanup sequences are listed in Table 3. Every combination is run, which gives 40 sweeps per kernel. The regime has 2,400 sweeps.
R6 The 60 highest-scoring chains of 3 distinct R3 passes. A chain's score is the sum of the instruction-count interaction (Equation 3), normalized by the canonical instruction count, over its consecutive distinct pairs; the diagonal contributes nothing. Ties are broken lexicographically. There are 2730 such chains, and this regime runs after the matrix has been computed from R3. It gives 3,600 sweeps. The matrix is built from exact instruction counts and carries no timing noise, so the chains do not inherit the noise of the timed matrix. R6 is still reported as a secondary result.

Deviations from the sweep design. LLVM 23.1.3 aborts a standalone `instcombine` on some functions, with a fixpoint error after one iteration. For those sweeps the engine retried once with the spelling `instcombine<`

Table 3 Cleanup sequences of R5 before and after `loop-vectorize`. Each row counts the sweeps that contain the sequence at that position, over all kernels.

Position	Sequence	Sweeps
Pre-pivot	none	300
Pre-pivot	instcombine	300
Pre-pivot	instcombine, jump-threading	300
Pre-pivot	jump-threading	300
Pre-pivot	jump-threading, simplifycfg	300
Pre-pivot	simplifycfg	300
Pre-pivot	simplifycfg, instcombine	300
Pre-pivot	simplifycfg, jump-threading	300
Post-pivot	none	480
Post-pivot	dce	480
Post-pivot	instcombine	480
Post-pivot	jump-threading	480
Post-pivot	simplifycfg	480

`no-verify-fixpoint>`, which removes the abort and nothing else. The default spelling was always tried first. Of the 3,488 completed sweeps that contain a standalone `instcombine`, 3,488 produced byte-identical IR under the two spellings, and 0 differed. 232 sweeps were run with the retry spelling, and the recorded pass sequence shows it.

Separately, `unify-loop-exits` terminated `opt` with a segmentation fault on 2 gsm kernels in an earlier campaign with the same pass set. The archived records of that campaign are kept with the other data, and their results are not reported. The sweep plan replaces that pass by `newgvn` in its R2 slot on every kernel before any sweep runs, so no R2 sweep crashes. The variant check reproduces the archived crashes on the `mem2reg`-only module, and it runs the replacement on the same module: 2 of 2 crashes reproduced, and 2 replacement runs succeeded.

4.3 Measured quantities

Each variant is optimized twice. The traced run uses `-print-changed`, `-debug-pass-manager`, and `-time-passes`, and it gives the redundancy counts and the instruction count. The clean run uses only `-time-passes`, so its wall time and pass time exclude the serialization that `-print-changed` performs. The two runs must produce identical IR, and a sweep that does not is recorded as failed. The sweep then reads four quantities. The instruction count N is the number of instructions of the optimized module, counted on its listing before lowering. The pass time P is the total in the `-time-passes` report of the clean run. The compile time C is the wall time of the clean `opt` process, measured around it. The sweep then lowers the optimized IR with `llc -O0 -filetype=obj`, links the object with the benchmark's libraries, and measures the `__text` section with `llvm-size -m`. The lowering level therefore affects the timing and the text size, and not the instruction count. The number of vectorized loops is counted from the `llvm.loop.isvectorized` metadata of the optimized module, because the remark stream can change the optimization that it reports on. The telemetry column named `scev_invalidation_count` is a proxy and is not used in the results.

4.4 IR-level checks

Each pass of the R3 set is applied to the canonical module, and then a second time to the result, with the same stage text that the sweeps use. The check is run on two canonical forms: the sweep canonical module, which applies `mem2reg`, `loop-simplify`, and `lcssa`, and the `mem2reg`-only module, which omits the `loop-simplify` and `lcssa` steps, rebuilt from each kernel's linked bitcode. For each form the check records whether the first application changes the bitcode, and whether the second application changes it again. A kernel where the second application leaves the bitcode unchanged is one where that pass is idempotent. The `instcombine` stage falls back to the spelling that the sweeps used after its fixpoint abort. The checks are reported in Table 6.

4.5 Timing

Timing is wall-clock time. A sweep's binary and its canonical binary are timed in one exclusive window. The order alternates between sweeps. Each binary receives 3 warm-up runs and then 15 timed runs, and its cost is the median of the timed runs. The canonical median measured in the same window is the reference J_0 for that sweep. Compilation runs under a readers–writer gate, so no compile job overlaps a timed window.

The telemetry column for the cost is named `exec_cycles_median` to keep the schema fixed, but its values are nanoseconds. Timing noise was measured on the sweeps whose optimized IR is identical to the canonical IR. Provided that lowering is deterministic, their paired deltas measure noise rather than code. Over 4,375 such sweeps (15.2% of the campaign), the median absolute paired delta was 0.73% of J_0, and the ninety-fifth percentile was 3.93%. These are descriptive statistics, and they set the noise floor for every timing result.

4.6 Refinement checks

For each sweep, the canonical IR and the optimized IR are read as LLVM textual listings. Each function is matched by name, and each block by label. A block pair is compared when both blocks lie in the fragment of Section 3.8 and have the same live-in names and widths. Every name defined in both blocks is checked with the query of Section 3.8, with a solver timeout of 5 seconds per name. The solver is Z3 [39]. A sweep's status is SAT if any name is violated, UNKNOWN if any query times out, UNSAT if every compared name refines, and not comparable if no name was compared. A block pair that falls outside the fragment is counted as not encodable and is never dropped silently.

For the Pareto-front sweeps, each query is also written as an `SMT-LIB 2` block and re-checked by the z3 command-line binary. That check is independent of the Python verdict, and it is not a separate proof. The optimized IR is rebuilt from the canonical bitcode by rerunning `opt` with the sweep's arguments and no traces. The rebuilt IR must have the same instruction count as the timed sweep. If it does not, the sweep is recorded as IR_NOT_REPRODUCED and receives no refinement status. An ERROR

status means that the check itself failed, for example because a tool exited with an error. Both outcomes are reported in the results instead of being excluded.

The encoder models each flag in Table 1, and a flag outside that table makes its block not encodable. Its agreement with an independent reference is checked exhaustively for widths up to 4 bits (Section 3).

4.7 Analysis

Every reported statistic is a deterministic function of the stored telemetry and of the IR-level checks. The instruction-count interactions are exact for each kernel. Their standard errors across kernels measure how much the kernels differ, and they are not noise estimates. An entry is marked when its mean exceeds two standard errors across kernels, which is a consistency mark, and the paper reports no hypothesis test. The timing interactions are divided by the canonical time and compared with the noise floor of Section 4.5. Shares, percentiles, the commutator formula, exact hypervolumes, and Shannon entropies in bits are computed as before.

A kernel's timing noise is estimated from its identical-IR sweeps as 1.4826 times the median absolute paired delta, which is the standard deviation of a zero-centered normal distribution with that median. The noise-aware variant of the ratio estimator floors each kernel's denominator at twice that noise.

Lowering pilot. A pilot lowers four kernels and four passes with `llc` at `-O2` and at `-O0`. The pilot is separate from the sweep telemetry, and Section 7.1 reports it.

Software. The software is Python `3.12` with `z3-solver 5.3.0.0`, LLVM 23.1.3 from Homebrew, and tectonic for the manuscript. Package versions are pinned in `requirements.txt`, and every table in the paper is generated from the stored outputs.

5 Results

We report what the telemetry supports, and we state where it does not support a claim. Every number below is a macro generated from the stored outputs (Section 4).

5.1 Campaign and measurement validity

All 28,800 sweeps have a complete telemetry row, and each row comes from the code path described in Section 4. Every sweep completed on its first attempt (0 failures). In an earlier campaign with the same pass set, `opt` crashed on 2 gsm kernels with `unify-loop-exits`. The pass is replaced by `newgvn` in its R2 slot on every kernel, so no R2 sweep crashes. Table 4 gives the sweeps per regime.

Benchmark outputs were compared with the canonical output of each kernel. Of the 28,800 sweeps, 28,796 produced the canonical output. The 4 exceptions are sweeps of `mibench/susan-smoothing`, and each writes an output image that differs from the canonical image in most of its bytes. Section 7.1 reports the investigation. For each of the exceptions, the same optimized module lowered at `-O2` and interpreted with `lli` gives the canonical output, so the difference appears only in the `-O0` lowering. These comparisons are reproduced in the replication package. We have not

Table 4 Sweeps per regime.

Regime	Purpose	Sweeps
R1	Standard and size levels	360
R2	Single passes	5,040
R3	Ordered pairs	13,500
R4	O3 prefixes	3,900
R5	Pass interference	2,400
R6	M_pass sequences	3,600
Total		28,800

Table 5 Largest instruction-count interactions over kernels, as a percentage of the canonical instruction count, with standard errors across kernels. The standard errors measure how much the kernels differ; the counts are exact.

First pass	Second pass	Mean (%)	SE (%)	Beyond 2 SE
loop-vectorize	loop-vectorize	-27.72	3.80	yes
gvn	gvn	+16.57	0.82	yes
early-cse	early-cse	+13.41	0.77	yes
early-cse	gvn	+13.17	0.75	yes
gvn	early-cse	+12.92	0.76	yes
instcombine	instcombine	+9.11	0.80	yes
simplifycfg	simplifycfg	+6.79	0.41	yes
early-cse	instcombine	+5.96	0.45	yes
gvn	instcombine	+5.51	0.47	yes
instcombine	early-cse	+5.31	0.52	yes
instcombine	gvn	+5.08	0.53	yes
jump-threading	simplifycfg	+4.61	0.27	yes

identified the cause, and we report these sweeps as exceptions rather than as matching outputs. For the bitcount kernel, the comparison was repeated under the corrected masking rule, and 480 of 480 sweeps matched. Each kernel runs on one input, so this agreement is evidence about those inputs only.

5.2 Instruction-count interactions

The instruction count is exact and is measured before lowering, so these results do not depend on the timing noise or on the lowering level. Figure 1 shows the tensor of instruction-count interactions over the 15 R3 passes, averaged over the kernels. Of its 225 ordered pairs, 116 have a mean that exceeds two standard errors across the kernels. The standard errors measure how much the kernels differ, because each kernel's count is exact. Table 5 lists the largest entries. The largest magnitude is 27.72% of the canonical instruction count, and it belongs to a self-pair. In Figure 1, rows are the first pass and columns the second. Positive values mean that the combined change of a pair is larger than the sum of its single changes. The diagonal holds the self-interactions, which are left blank in the figure and reported in Table 5.

Several of the largest entries involve passes that remove redundant instructions. The two orders of `early-cse` and `gvn` are both positive, which is what overlapping removals would produce: when the two passes remove the same redundant instructions, the pair removes fewer than the sum of its single removals.

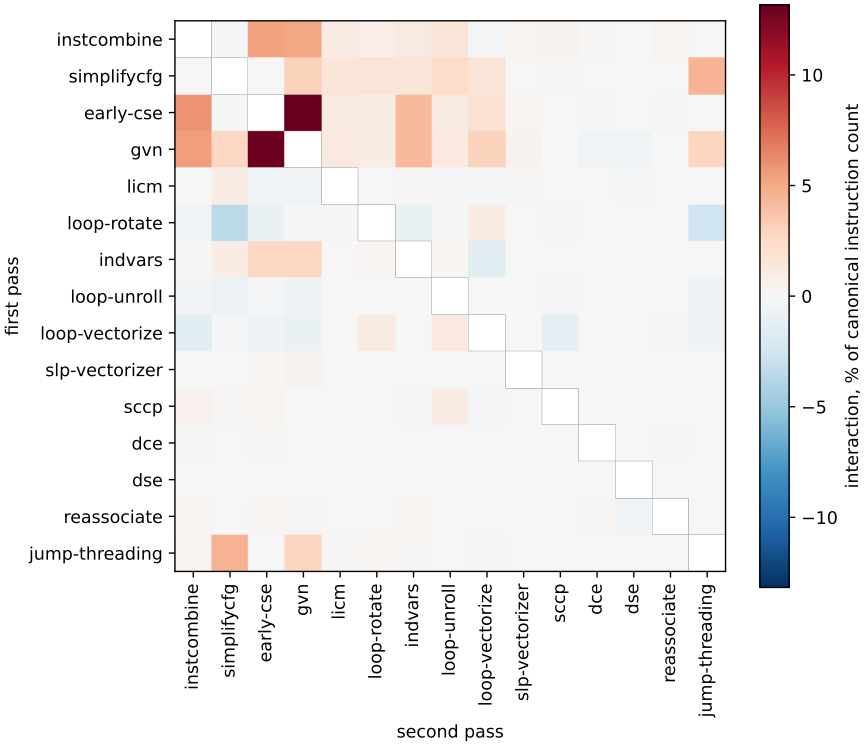


Fig. 1 Instruction-count interaction of the 15 R3 passes, averaged over kernels.

Self-interactions. The identity of Remark 1 holds wherever a second application leaves the bitcode unchanged. Section 5.3 counts 331 such kernel-pass cases, and the identity holds exactly in all of them, with 0 mismatches. On those kernels, a positive self-interaction marks a pass whose first application removes instructions, and a negative one marks a pass whose first application adds them. The sign of a self-interaction is therefore not evidence of a second effect, and the self-pairs of `gvn`, `early-cse`, and `loop-vectorize` in Table 5 are read this way. Where the second application changes the IR again, the identity does not hold, and the self-interaction mixes the second change with the first.

5.3 Canonical form and repeated application

Table 6 reports the IR-level checks over 60 kernels. Each pass of the R3 set is applied to a module, and then again to the result. The table compares two canonical forms as a controlled ablation of the canonicalization: the sweep canonical form, which applies `mem2reg`, `loop-simplify`, and `lcssa`, and the `mem2reg`-only form, which omits the `loop-simplify` and `lcssa` steps.

On the `mem2reg`-only form, every loop pass of the set changes every kernel, and the smallest count among the five loop passes is 60. On the sweep canonical form,

Table 6 IR-level checks over 60 kernels. Changed counts the kernels where the pass changes the module, on the sweep canonical form and on the mem2reg-only form. Repeated counts the kernels where a second application changes it again, out of those where the first changed it.

Pass	Changed	Repeated	Mem2reg: changed	Mem2reg: repeated
instcombine	60	30	60	30
simplifycfg	60	23	60	22
early-cse	60	53	59	46
gvn	60	27	60	20
licm	58	23	60	25
loop-rotate	60	32	60	31
indvars	59	42	60	42
loop-unroll	24	21	60	23
loop-vectorize	57	44	60	44
slp-vectorizer	31	19	32	20
sccp	60	26	60	26
dce	27	19	25	20
dse	24	19	22	20
reassociate	60	24	60	22
jump-threading	60	27	60	25

the smallest count is 24, for `loop-unroll`, which changes 24 of 60 kernels there. So the canonical form changes the effect of `loop-unroll` in particular. For every other pass, the two forms differ by at most 3 kernels.

Repeated application changes the IR again in 38% to 88% of the kernels where the first application changed it. So a pass that changes the module is not, in general, idempotent on these kernels. The identity of Remark 1 is therefore used only where a second application is unchanged, and the 331 such cases are the ones on which it is tested.

5.4 Timing interactions

The timing measure is secondary, because its noise is large relative to the interactions. The median absolute paired delta of the identical-IR sweeps is 0.73% of the canonical cost, and the ninety-fifth percentile is 3.93%. Of the 225 entries of the normalized timing interaction (Equation 4), 26 exceed two standard errors (Table 7). Under a normal approximation, two standard errors flag about five percent of entries by chance, so this count is larger than chance alone would give. The largest magnitude is 3.40% of the canonical cost, which lies below the ninety-fifth percentile of the noise. The entries that pass the test are therefore small, and we do not treat them as established interactions.

The single-effect ratio baseline (Equation 5) puts most of its largest values on self-pairs, with standard errors as large as their means, and none of its five largest entries passes the two-standard-error test (Table 8). Table 9 compares it with the canonical-cost normalized estimator on the same pairs.

The commutators are antisymmetric by construction (Table 10). Of the 225 pairs, 12 differ from zero by more than two standard errors, and the largest mean magnitude is 0.0279 of the canonical cost, which lies below the ninety-fifth percentile of the timing noise.

Table 7 Largest pairwise interaction entries over kernels, as a percentage of the canonical cost, with standard errors. Each entry is the mean of $(\Delta_{ij} - \Delta_i - \Delta_j)/J_0$ over kernels.

First pass	Second pass	Mean (%)	SE (%)	Beyond 2 SE
instcombine	jump-threading	+3.396	1.215	yes
instcombine	instcombine	+3.274	1.385	yes
gvn	instcombine	+3.061	1.362	yes
jump-threading	loop-rotate	-2.962	0.743	yes
simplifycfg	loop-rotate	-2.926	0.809	yes
simplifycfg	instcombine	+2.901	1.270	yes
loop-rotate	instcombine	+2.837	1.467	no
loop-rotate	jump-threading	-2.789	0.876	yes
jump-threading	instcombine	+2.679	1.276	yes
gvn	loop-rotate	-2.625	0.745	yes
instcombine	loop-rotate	+2.610	1.289	yes
loop-rotate	simplifycfg	-2.591	0.815	yes

Table 8 Largest entries of the ratio estimator, $\mathbf{M}^{\text{ratio}}_{\text{pass}}$ (Equation 5), with standard errors. An entry is marked when its magnitude exceeds two standard errors.

First pass	Second pass	Mean	SE	Beyond 2 SE
reassociate	reassociate	-3.7198	3.6405	no
instcombine	loop-rotate	-2.7512	2.7181	no
loop-vectorize	loop-vectorize	-2.2911	2.3976	no
slp-vectorizer	slp-vectorizer	-1.2466	0.8554	no
instcombine	dce	-0.9200	0.9568	no
loop-rotate	gvn	-0.6911	0.2307	yes
instcombine	early-cse	-0.6766	0.5147	no
loop-rotate	jump-threading	-0.6730	0.1786	yes
jump-threading	loop-rotate	-0.6389	0.1667	yes
gvn	loop-rotate	-0.6141	0.2177	yes
instcombine	dse	-0.5888	0.3660	no
reassociate	licm	+0.5722	0.3749	no

Table 9 The same pass pairs under the ratio estimator (Equation 5) and under the timing interaction normalized by the canonical cost (Equation 4). The ratio columns are dimensionless. The interaction columns are percentages of the canonical cost.

First pass	Second pass	Ratio mean	Ratio SE	Mean (%)	SE (%)
dse	dse	+0.42	0.27	+0.97	0.76
licm	licm	+0.37	0.71	+0.44	0.77
early-cse	early-cse	+0.05	0.30	+0.58	0.65
gvn	gvn	+0.46	0.45	-0.96	1.31
sccp	sccp	-0.27	0.35	-0.36	0.52
jump-threading	loop-vectorize	-0.19	0.10	-0.88	0.78

5.5 Cleanup sequences around loop-vectorize

Table 11 gives the timing effect of each cleanup sequence on `loop-vectorize`. Of the 11 sequences, 2 have a mean effect beyond two standard errors. Single kernels do show effects larger than their noise, and up to 38 of the 60 kernels exceed two noise standard deviations for one sequence. The identical-IR noise estimate covers binaries that are identical, so it does not cover the differences in code layout and alignment that

Table 10 Largest mean Lie-bracket commutators over kernels, in units of the canonical cost, with standard errors.

First pass	Second pass	Mean	SE	Beyond 2 SE
loop-rotate	loop-vectorize	-0.0279	0.0131	yes
loop-vectorize	loop-rotate	+0.0279	0.0131	yes
early-cse	loop-rotate	+0.0153	0.0055	yes
loop-rotate	early-cse	-0.0153	0.0055	yes
licm	loop-rotate	-0.0147	0.0070	yes
loop-rotate	licm	+0.0147	0.0070	yes
simplifycfg	indvars	-0.0138	0.0092	no
indvars	simplifycfg	+0.0138	0.0092	no
gvn	licm	+0.0133	0.0061	yes
licm	gvn	-0.0133	0.0061	yes
instcombine	loop-vectorize	+0.0090	0.0070	no
loop-vectorize	instcombine	-0.0090	0.0070	no

Table 11 Interference of cleanup sequences with `loop-vectorize`, as a percentage of the canonical cost. Mean, standard error, and median are over kernels. The last column counts kernels whose effect exceeds two noise standard deviations of that kernel.

Position	Sequence	Mean (%)	SE (%)	Median (%)	Beyond noise
after	dce	+0.11	0.24	-0.07	6
after	instcombine	-2.75	1.13	-0.98	23
after	jump-threading	-0.35	0.98	+0.23	28
after	simplifycfg	-2.14	1.69	-0.01	38
before	instcombine	-1.79	0.65	-0.77	24
before	instcombine+jump-threading	-0.48	1.01	+0.17	32
before	jump-threading	-0.11	0.96	-0.48	37
before	jump-threading+simplifycfg	-1.92	1.56	-0.21	32
before	simplifycfg	-1.95	1.65	-0.02	29
before	simplifycfg+instcombine	-2.29	1.69	-0.33	34
before	simplifycfg+jump-threading	-2.38	1.63	-0.30	35

distinct binaries can show. We therefore read single-kernel effects of a few percent as unresolved.

5.6 Redundant passes and compile-time waste

A pass execution is redundant when the IR it leaves unchanged. Across all sweeps the mean redundant-pass ratio is 0.633. Table 12 shows it per regime. The standard levels have the highest ratio, 0.844, and the `-O3` prefixes have 0.745. Figure 2 shows the ratio and the opcode entropy along the prefixes. Charging each redundant execution with its share of its sweep's pass time (Equation 8) gives 13.40 seconds in total across all sweeps. That share is an assumption, because the telemetry records the pass time of each sweep and not of each execution.

5.7 Pareto fronts and hypervolume

Across the 60 kernels, the three-objective fronts contain 1,789 non-dominated points. Table 13 gives the hypervolume per suite at the reference point of the canonical build, in units of 10^{-6}. The mean over all kernels is 83.79 in those units. These values are

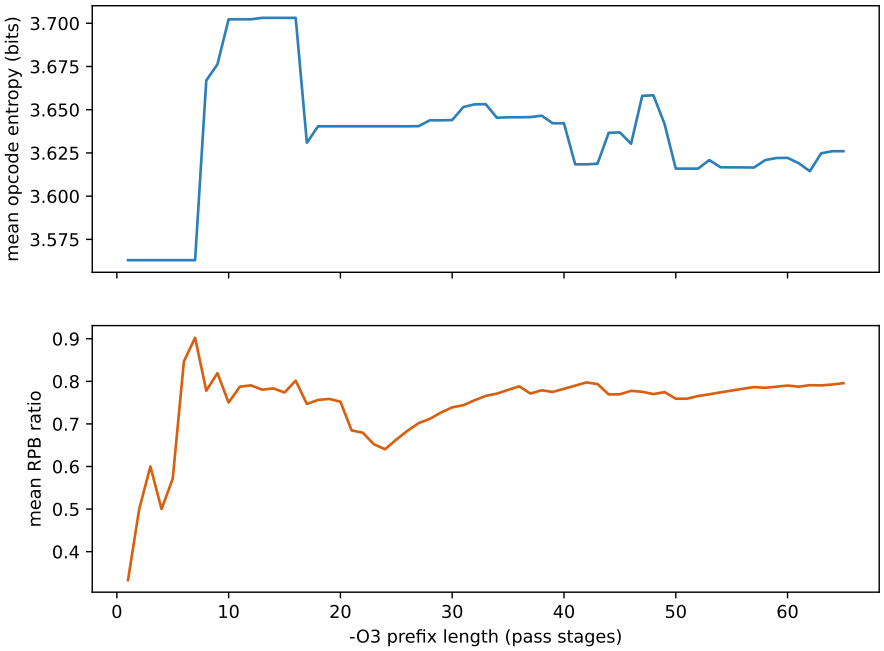


Fig. 2 Mean opcode entropy (top) and mean redundant-pass ratio (bottom) along the stages of the `-O3` pipeline.

Table 12 Redundant pass events and estimated compile-time waste per regime. The ratio is the mean over sweeps of the fraction of pass executions that -print-changed reports as unchanged. Waste is the sum over sweeps of compile time multiplied by that ratio.

Regime	Purpose	Mean ratio	Waste (s)
R1	Standard and size levels	0.844	0.36
R2	Single passes	0.622	1.70
R3	Ordered pairs	0.629	6.41
R4	O3 prefixes	0.745	2.40
R5	Pass interference	0.568	1.03
R6	M_pass sequences	0.559	1.49

small because the reference point is the canonical build, and the objective gains over it are small. Figure 3 shows how the hypervolume changes with the reference point. The compile reference is fixed, so the hypervolume is a reference-dependent summary. The speed objective uses the timing, and the compile objective uses the optimizer wall time, so both carry the limits of Section 7.1.

Of the 1,789 front members, 45.6% are UNSAT, 50.1% are not comparable, and 4.4% are SAT. The study therefore does not establish that every Pareto schedule refines its canonical build, and the paper does not claim it.

Table 13 Exact three-objective hypervolume at the reference point (0.5, 0.5, 0.5), in units of 10^{-6}, and Pareto-front size, averaged over the kernels of each suite.

Suite	Kernels	Mean hypervolume ($\times 10^{-6}$)	Mean front size
mibench	20	215.41	24.4
polybench	30	18.42	32.5
tsvc_coremark	10	16.63	32.7

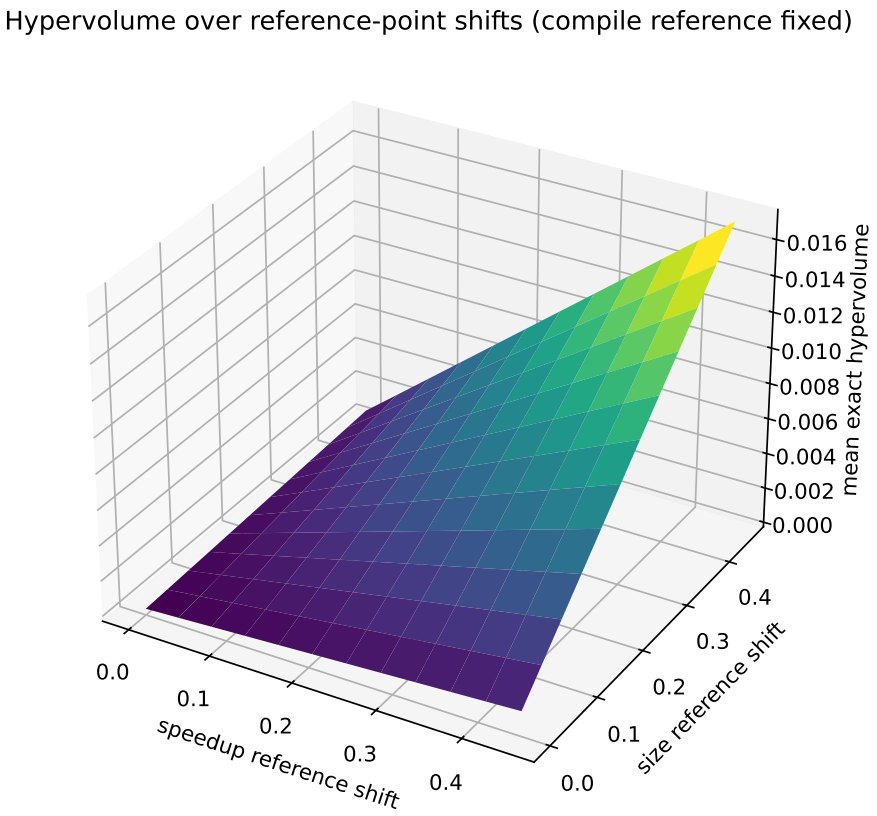


Fig. 3 Mean exact hypervolume of the per-kernel fronts as the speedup and size reference points are lowered.

5.8 Refinement outcomes

Table 14 gives the block-local verdict of each sweep per regime. Over all sweeps, 56.6% are UNSAT (16,293 sweeps), 10.4% are SAT (2,999), and 33.0% are not comparable (9,508). No sweep is UNKNOWN or ERROR, and every sweep's optimized IR was reproduced by re-running `opt` (0 failures).

The verdicts are limited by coverage. Of the 3,430,824 matched block pairs, 26.7% lie inside the fragment of Section 3.8, and the rest contain an instruction the fragment does not model. Every not-comparable sweep has matched labels, but none of its matched blocks lies in the fragment. Within the encodable blocks, 544,249 names

Table 14 Block-local refinement outcome per sweep, as a percentage of the sweeps in each regime. Failed counts tool errors and optimized IR that did not reproduce.

Regime	Sweeps	UNSAT	SAT	Unknown	Not comparable	Failed
R1	360	31.7%	0.0%	0.0%	68.3%	0.0%
R2	5,040	91.9%	3.3%	0.0%	4.8%	0.0%
R3	13,500	61.3%	15.4%	0.0%	23.3%	0.0%
R4	3,900	34.1%	0.5%	0.0%	65.4%	0.0%
R5	2,400	39.3%	9.2%	0.0%	51.5%	0.0%
R6	3,600	27.9%	14.2%	0.0%	57.8%	0.0%

refine and 13,523 are violated. The share of not-comparable sweeps is highest in R1 and R4 and lowest in R2, as Table 14 shows. The analysis does not classify which instructions cause the exclusion in each case.

Every SAT witness falls into exactly one of three categories, and the categories cover all 13,523 SAT witnesses. Category A is a flag gain: the optimized instruction differs from its canonical instruction only in wrap or exactness flags, and it carries at least one flag that the canonical one lacks. Category B is a flag loss only. Category C is every other witness, which keeps its text or changes it in some way other than flags. There are 7,812 witnesses in A, 49 in B, and 5,662 in C. Most SAT answers (7,861 of 13,523, 58.1%) name an instruction whose only change is in its flags. Of the 2,999 SAT sweeps, 1,577 (52.6%) contain a witness in category A. Across all 13,523 SAT witnesses, the optimizer added 8,678 flags: `nuw` 7,430 times, `nsw` 774, `nneg` 458, and `samesign` 16. No witness gained an `exact` or `disjoint` flag. The other witnesses changed the instruction in other ways (4,743, of which 8 differ only in operand names), kept its text so that an upstream value explains the difference (919), or lost a flag (49). A witness that only loses a flag cannot be a poison counterexample, so its answer also depends on an upstream value.

Replaying each sweep one stage at a time locates the first stage that carries each gained flag on the same name. The replay reproduces the optimized IR exactly in 1,577 of 1,577 replayed sweeps. Of the 8,464 located flag introductions, `instcombine` accounts for 5,986 (70.7%), `sccp` for 1,799 (21.3%), and `indvars` for 404 (4.8%). The other 275 come from five passes. For 214 introductions, no stage kept the name, so they are not located. These counts identify the passes that add the flags. They do not show why each flag is justified.

One justification is inspected in full. Sweep `K37-R2-018` runs `indvars` alone on the `patricia` kernel. Its SAT witness is `%157` in block `156` of `pat_insert`, where the canonical instruction `add nsw i32 %.3, 1` becomes `add nuw nsw i32 %.3, 1`. The counterexample is `%.3 = -1`, for which the unsigned increment wraps. The value `%.3` is a loop counter, however. Its phi takes the value `1` on entry and `%157` on the back edge, and `%157` is the only increment, so the counter never takes a negative value. The block-local checker treats `%.3` as a free input and admits `-1`, which the loop cannot produce. The same fact also justifies the `samesign` flag that `indvars` gives the loop-header comparison, which contains a phi and so lies outside the checker's fragment. This case is a false alarm of the block-local check, not a miscompilation. We have not inspected the other sweeps, and this case does not show that they behave the same way.

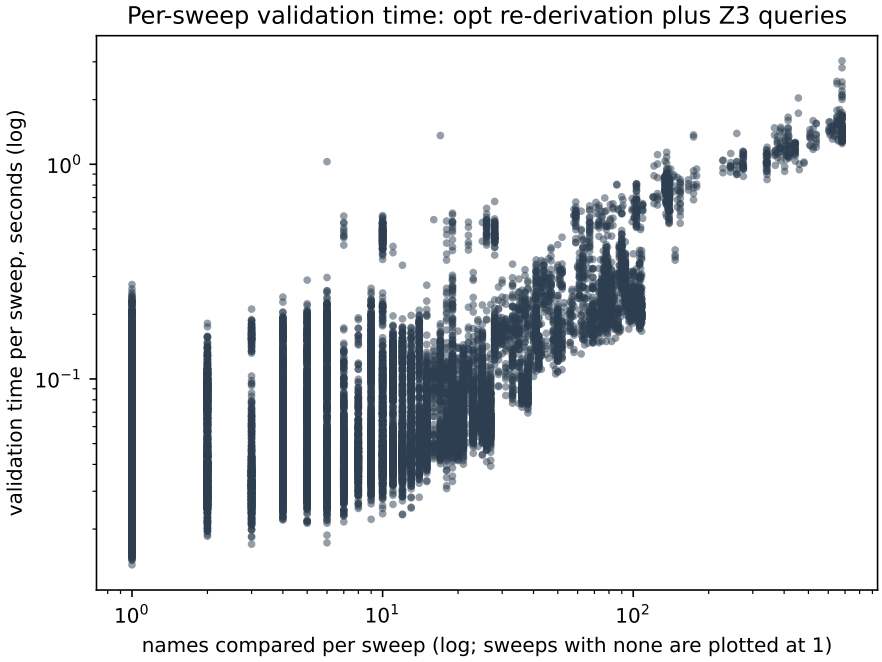


Fig. 4 Validation time per sweep against the number of names compared; sweeps with no compared name are plotted at one.

A coarser count compares the number of flag tokens in each optimized module with the canonical module. By that count, 75.2% of the SAT sweeps (2,255 sweeps) gain a flag. We do not use it. It rises whenever a flagged instruction appears anywhere in the module, including code that no witness names, so it cannot separate inference from copying.

Of the 28,800 sweeps, 28,796 matched the canonical output on the one input their kernel runs. None of the 4 exceptions is a SAT sweep: all of them are UNSAT on their encodable blocks, so the block-local verdict does not account for their output difference. The UNSAT verdicts are proofs only for the blocks that were compared, and the block-local check does not cover floating-point or memory operations outside its fragment, so the verdicts do not show that the rest of each program is unchanged. Figure 4 shows the validation time per sweep. It includes the re-derivation of the optimized IR with `opt`, and it grows with the number of names compared.

Every refinement query of the Pareto-front sweeps is also exported as an `SMT-LIB 2` certificate. The export covers 1,789 sweeps and 17,204 queries, of which 16,604 are UNSAT and 600 are SAT. The z3 command-line binary, run independently on the exported text, returned the same verdict for 17,204 queries. The number of sweeps whose status disagrees with the telemetry is 0. The certificates are in the repository, one gzip-compressed file per sweep.

6 Discussion

The introduction asked four questions. The answers are partial, and the first and the third carry the most weight.

Pass interactions. The instruction-count interactions are exact, large, and structured. Of the 225 ordered pairs of the 15 R3 passes, 116 have a mean beyond two standard errors across kernels. The largest entries are self-pairs and pairs of passes that remove the same redundant code. Where a second application of a pass leaves the bitcode unchanged, the self-interaction equals minus the pass's single effect, and that identity holds exactly in all 331 cases tested. The timing interactions add little to this picture. 26 of 225 normalized timing interactions pass the two-standard-error test. That is more than chance alone would give, but every entry is smaller than the noise floor, so we do not count them as established.

Canonical form. On the `mem2reg`-only form, the five loop passes change every kernel. On the sweep canonical form, the smallest count among them is 24, for `loop-unroll`. The canonical form therefore matters for `loop-unroll`, so the choice of canonical form is part of the result and not only a setup detail. For the other passes, the two forms differ by at most 3 kernels.

Redundant executions. Redundancy is common. On average, 0.633 of the pass executions in these sweeps leave the IR unchanged. Charged with their share of the pass time, the redundant executions consume 13.40 seconds across all sweeps. Only a small part of the optimizer's wall time is pass time, so the wall time is not used for the waste.

Pareto structure. The fronts contain 1,789 non-dominated points. Their hypervolume depends on the reference point, so a value is meaningful only with its reference. Two of the three objectives depend on timing measurements, which carry noise. The speed objective uses the timing, and the compile objective uses the optimizer wall time.

Verification. The block-local checker decides refinement for the blocks in its fragment, which are 26.7% of the matched block pairs. Within those blocks, 544,249 names refine and 13,523 are violated. In 52.6% of the SAT sweeps, the candidate counterexample names an instruction to which the optimizer added a flag. The remaining candidates involve other changes or upstream values. The benchmark outputs match the canonical outputs in all but 4 of 28,800 sweeps, on the one input each kernel runs. None of the exceptions is a SAT sweep. For each of them, the difference disappears under `-O2` lowering. That is consistent with the candidates being unreachable on those inputs, but it is not a proof of correctness. The one candidate inspected in full is excluded by an invariant of its loop, not by any single block, so SAT answers are reported as candidates for inspection, not as miscompilations.

Four implications follow. First, a measure that is exact for each kernel, such as the instruction count, should be preferred to a noisy one whenever the question allows it. Second, the canonical form is a parameter of the study, and it should be reported and tested, as Section 5.3 does for the loop passes. Third, block-local verification of optimized code needs facts from the surrounding function, such as value ranges, before its SAT answers can be trusted without inspection. Fourth, benchmark output can depend on timing. A canonical control failed on the bitcount kernel, and the kernel's ranking

lines were masked under a rule recorded with the kernel. Any campaign that compares outputs should include such a control, and it should also test the lowering level, as the exceptions of Section 5 show.

Future work follows from these limits. The first item is a sensitivity analysis of the canonical form for every pass, not only the loop passes. The second is a measurement method with less noise, so that the timing interactions can be measured below the noise floor. The third is a function-level encoding with range facts, which would reduce the SAT answers that depend on context. The fourth is a compile-time objective based on pass time, which would change the fronts of Section 5. Larger kernel sets, a second machine, and whole-program benchmarks would test whether these results hold beyond small loop kernels.

7 Conclusion

We measured the interactions among the passes of the LLVM new pass manager on 60 kernels, over 28,800 pipeline variants. The main measure is the instruction count of the optimized IR, which is exact and does not pass through the backend. Of its pairwise interactions, 116 of 225 are consistent across kernels, and the self-interactions follow an identity that holds in every tested case. Timing resolves few of these interactions, and the ones it flags lie below the noise floor. Redundant executions are common. The refinement checker decides refinement for a minority of the blocks it examines, and in 52.6% of its SAT sweeps the optimizer added a flag to the instruction that the SAT answer names. The one such case inspected in full is a false alarm of the block-local check. 4 sweeps produce output that differs from the canonical output under the `-O0` lowering, and we have not explained the difference. These are the claims the data support. They are bounded by one compiler release, one lowering level for the timing results, and one input per kernel.

7.1 Threats to validity

Measures. The main results use the instruction count of the optimized IR. That count measures code size at the IR level, and it says nothing about running time. The timing results carry measurement noise, and the speed objective of the Pareto analysis inherits it. The compile objective uses the optimizer wall time, which includes process start-up and trace output. Changing that objective to pass time would change the fronts, and the paper does not redo them.

Lowering. The timing results and the text sizes use an optimized module lowered with `llc -O0`, so they exclude backend optimization. The instruction counts do not depend on lowering. A pilot (Section 4) compares the size change of four passes on four kernels at `-O2` and at `-O0`. It gives 12 nonzero size changes at `-O0` out of 16 cases, against 10 at `-O2`, and 2 cases change sign between the two levels. The `-O0` lowering was chosen to keep backend optimization out of the size and timing results. It has a cost of its own, because it can change program output. The four output exceptions of Section 5 are sweeps of `susan-smoothing` whose `-O0` binaries differ from the canonical binary, and for each of them the same optimized IR lowered at `-O2` gives

the canonical output. The pilot used object files, not linked executables, so its sizes are a proxy for the sweeps' binaries.

Canonical form. The sweeps use the sweep canonical form, which applies `mem2reg`, `loop-simplify`, and `lcssa`. Section 5.3 shows that the loop passes depend on this choice. On the `mem2reg`-only form, every loop pass changes every kernel, whereas on the sweep canonical form `loop-unroll` changes only 24 of 60 kernels. The other passes are less sensitive, and the two forms differ by at most 3 kernels for each of them. A pass that depends on a loop form it does not find may do nothing on these inputs, and the study does not separate that case from a pass that has no effect.

Timing. Paired windows remove the bias of unpaired baselines, and the identical-IR runs bound the noise of identical code. They do not bound the differences between distinct binaries. Code layout and alignment can change the running time of two binaries that compute the same thing, and the identical-IR runs cannot show that effect. For this reason the timing comparisons count exceedances of the noise floor but do not treat them as interference. **Benchmarks and inputs.** The 60 kernels are small, and most of them are loop nests or control-flow kernels. Each kernel runs on one input. The output agreement reported in Section 5 is therefore evidence about those inputs only, and a miscompilation that appears only on other inputs would not be detected. The bitcount kernel shows that benchmark output can depend on timing. Its ranking lines were masked after the control check failed, and the rule is recorded with the kernel. 4 sweeps of `susan-smoothing` produce output images that differ from the canonical images under the `-O0` lowering, so the output comparison is not a complete check, and the paper reports those sweeps as exceptions.

One compiler release. All results use LLVM 23.1.3. 8 canonical passes are rejected by this release, and 8 substitutes replace them. An earlier campaign recorded 2 crashes of one single-pass sweep type, `unify-loop-exits`, on specific kernels. The current campaign replaces that pass by `newgvn` in its slot on every kernel before any sweep runs. The size levels use forced function attributes, because the release rejects the `Os` and `Oz` pipeline names. A standalone `instcombine` aborts on some functions. The spelling that skips the fixpoint check gave identical IR on 3,488 of the 3,488 completed sweeps that contain the pass, and 0 sweeps differed. It is still a different spelling of the same pass. All of these choices could change the results in another release.

Scope of the refinement proofs. The UNSAT answers are proofs for block pairs in the fragment of Section 3.8, for names defined in both blocks, and for non-poison live-ins. They say nothing about memory, calls, floating point, vectors, control flow inside the block, or facts that hold only in the surrounding function. Only 26.7% of matched block pairs lie in the fragment, so most of each program is outside the proofs. A SAT answer is a candidate counterexample, and the paper reports it as one. The encoder's agreement with the reference semantics is checked exhaustively for small widths, which is finite evidence for the lemma of Section 3.

Separate evidence. The timing measurements, the refinement checks, and the output comparisons are separate evidence. A sweep whose optimized IR refines its canonical IR can still be slower, and a faster sweep is not thereby correct. The paper

draws conclusions about correctness only from the refinement results and the output comparisons, and about cost only from the instruction counts and the timings.

Estimators. The single-effect ratio baseline is a diagnostic only. Its denominator is noise-sized for 38.1% of the kernel-entry pairs, and the regime R6 chains are selected from the instruction-count matrix instead, so they do not inherit that instability. The pass-time waste assumes that every execution costs the same share of its sweep's pass time. The hypervolume depends on the reference point, as Figure 3 shows, and the canonical reference was fixed before the analysis.

Data and Code Availability

The source code, the unit tests, the telemetry, the figures, the manuscript sources, the benchmark manifests, and the curated bibliography (`manuscript/references_curated.bib`) are available on GitHub at **[GitHub URL to be inserted before submission]**. The sweep dataset is `data/telemetry/master_sweeps_28800.csv`, with one row per sweep. The other outputs in `data/telemetry/` are the refinement analysis, the SAT-witness attribution, the Pareto fronts, the pass transition matrix, the lowering pilot, the pass checks, and the output exceptions. The SMT-LIB v2.6 certificates for the Pareto-front sweeps are in `data/smt2_certificates/`. The file `manuscript/generated/provenance.json` records the `SHA-256` checksum of each input that `src/task6_manuscript_author.py` reads, and that script regenerates the macros and tables the text uses. The four output exceptions are reproduced by `src/investigate_output_exceptions.py`. The instruction-count figure is drawn by `src/figure_instcount.py`, and the other figures by `src/task5_plot_generator.py`; the checksum file does not cover the figures.

The benchmark suites evaluated in this study are open-source third-party software and are not redistributed directly. The replication repository contains automated acquisition and build manifests that record the upstream origin and the cryptographic checksum of every kernel, together with the build scripts. The ingested benchmark sources are PolyBench/C `4.2.1` (beta), obtained from the SourceForge mirror recorded in the manifest; MiBench v1.0, obtained from the per-category archives published by the University of Michigan; the TSVC vectorization suite from the LLVM test suite at the revision recorded in `data/benchmarks/tsvc_coremark/MANIFEST.json`; and CoreMark at the revision recorded in the same file. Five of the six CoreMark checksums in the upstream registry match the ingested source files. The sixth, `coremark.h`, differs by one comment word, because the upstream project changed a comment without regenerating its checksum registry.

Funding, Competing Interests, and Author Contributions

Corresponding Author: Kanagaraj R (kanagaraj.r@vit.ac.in).

Funding: The authors attest that no external financial support, grant, fee, payment, or other benefit was provided to or received by them for the research, design, or publication of this work.

Competing Interests: The authors attest that nothing of financial or other value was provided to or taken from any person or organization in connection with this work. They declare no financial, personal, or professional competing interests that could have influenced the design, execution, or interpretation of this study.

Author Contributions: Devarsh Patel and Darshil Vaid conceived the research trajectory, formulated the theoretical framework and mathematical pass algebra, developed the multi-core execution engine and SMT translation validator, analyzed the experimental telemetry, and co-authored the final manuscript. Devarsh Patel and Darshil Vaid reviewed, refined, and approved the final manuscript. **[Kanagaraj R's contribution, and confirmation of approval of the final manuscript, to be stated by the authors.]**

Ethics Approval: Not applicable. This study involves exclusively deterministic computational benchmarking and formal logic verification on standard user-space open-source software, with zero human subjects or animal testing.

Consent to Participate: Not applicable.

Consent for Publication: Not applicable. The manuscript contains no individual's data.

References

[1] Lattner, C., Adve, V.: LLVM: a compilation framework for lifelong program analysis & transformation. In: International Symposium on Code Generation and Optimization, 2004. CGO 2004., pp. 75–86. IEEE (2004). https://doi.org/10.1109/cgo.2004.1281665

[2] Ashouri, A.H., Shirahmad Gale Bagi, S., Satheeskumar, K., Srikanth, T., Zhao, J., Saidoun, I., Wang, Z., Chan, B., Czajkowski, T.S.: Protean Compiler: An Agile Framework to Drive Fine-grain Phase Ordering. ACM Transactions on Architecture and Code Optimization **23**(3), 1–26 (2026). https://doi.org/10.1145/3831596

[3] Han, R., Kim, H.: Exponentially Expanding the Phase-Ordering Search Space via Dormant Information. In: Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction, pp. 250–261. ACM (2024). https://doi.org/10.1145/3640537.3641582

[4] Xie, Z., Wen, M., Qiu, S., Jin, H.: Validating JVM Compilers via Maximizing Optimization Interactions. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4, pp. 345–360. ACM (2024). https://doi.org/10.1145/3622781.3674188

[5] Seeker, V., Cummins, C., Cole, M., Franke, B., Hazelwood, K., Leather, H.: Revealing Compiler Heuristics Through Automated Discovery and Optimization. In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 55–66. IEEE (2024). https://doi.org/10.1109/

cgo57630.2024.10444847

[6] Lenfers, J., Spehr, S., Dieckmann, J., Jansen, J., Lücke, M.P., Gorlatch, S.: Schedgehammer: Auto-tuning Compiler Optimizations beyond Numerical Parameters. In: Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction, pp. 119–130. ACM (2026). https://doi.org/10.1145/3771775.3786282

[7] VenkataKeerthy, S., Jain, S., Kalvakuntla, U., Gorantla, P.S., Chitale, R.S., Brevdo, E., Cohen, A., Trofin, M., Upadrasta, R.: The Next 700 ML-Enabled Compiler Optimizations. In: Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction, pp. 238–249. ACM (2024). https://doi.org/10.1145/3640537.3641580

[8] Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G., Leather, H.: LLM Compiler: Foundation Language Models for Compiler Optimization. In: Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction, pp. 141–153. ACM (2025). https://doi.org/10.1145/3708493.3712691

[9] Gassmann, T., Chaliasos, S., Sotiropoulos, T., Su, Z.: Evaluating Compiler Optimization Impacts on zkVM Performance. In: Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 697–714. ACM (2026). https://doi.org/10.1145/3779212.3790159

[10] Santos, F.F.D., Carro, L., Vella, F., Rech, P.: Assessing the Impact of Compiler Optimizations on GPUs Reliability. ACM Transactions on Architecture and Code Optimization **21**(2), 1–22 (2024). https://doi.org/10.1145/3638249

[11] Alladi, S., Ros, A., Jimborean, A.: Enabling Automatic Compiler-Driven Vectorization of Transformers. In: 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 319–333. IEEE (2026). https://doi.org/10.1109/cgo68049.2026.11395226

[12] Murtovi, A., Georgakoudis, G., Parasyris, K., Liao, C., Laguna, I., Steffen, B.: Enhancing Performance Through Control-Flow Unmerging and Loop Unrolling on GPUs. In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 106–118. IEEE (2024). https://doi.org/10.1109/cgo57630.2024.10444819

[13] Zhi, Y., Cao, Y., Dai, J., Han, X., Pu, J., Wu, Q., Cheng, S., Cai, M.: LOOPRAG: Enhancing Loop Transformation Optimization with Retrieval-Augmented Large Language Models. In: Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 1113–1135. ACM (2026). https://doi.org/10.1145/3779212.3790183

[14] Lopes, N.P., Lee, J., Hur, C.-K., Liu, Z., Regehr, J.: Alive2: bounded translation validation for LLVM. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, pp. 65–79. ACM (2021). https://doi.org/10.1145/3453483.3454030

[15] Kwon, H., Shin, S., Lee, J.M., Youm, H., Song, S., Kim, S., Jung, H., Lee, S., Kim, H.: Compiler-Runtime Co-operative Chain of Verification for LLM-Based Code Optimization. In: 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 617–629. IEEE (2026). https://doi.org/10.1109/cgo68049.2026.11395240

[16] Italiano, D., Cummins, C.: Finding Missed Code Size Optimizations in Compilers using Large Language Models. In: Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction, pp. 81–91. ACM (2025). https://doi.org/10.1145/3708493.3712686

[17] Yuan, S., He, J., Xie, Y., Wang, F., Zhao, J.: Post-Link Outlining for Code Size Reduction. In: Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction, pp. 154–166. ACM (2025). https://doi.org/10.1145/3708493.3712692

[18] Saito, Y., Sakamoto, K., Washizaki, H., Fukazawa, Y.: Multiple Function Merging for Code Size Reduction. ACM Transactions on Architecture and Code Optimization **22**(1), 1–26 (2025). https://doi.org/10.1145/3702000

[19] Cugurovic, M., Prokopec, A., Spasojevic, B., Jovanovic, V., Vujošević Janičić, M.: GraalMHC: ML-Based Method-Hotness Classification for Binary-Size Reduction in Optimizing Compilers. In: Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction, pp. 1–13. ACM (2026). https://doi.org/10.1145/3771775.3786276

[20] Mu, P., Mavrogeorgis, N., Vasiladiotis, C., Tsoutsouras, V., Kaparounakis, O., Stanley-Marbell, P., Barbalace, A.: CoSense: Compiler Optimizations using Sensor Technical Specifications. In: Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction, pp. 73–85. ACM (2024). https://doi.org/10.1145/3640537.3641576

[21] Schwarz, T., Kamm, T., Engelke, A.: TPDE: A Fast Adaptable Compiler Back-End Framework. In: 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 427–439. IEEE (2026). https://doi.org/10.1109/cgo68049.2026.11395208

[22] Engelke, A., Schwarz, T.: Compile-Time Analysis of Compiler Frameworks for Query Compilation. In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 233–244. IEEE (2024). https://doi.org/10.1109/cgo57630.2024.10444856

[23] Titzer, B.L.: Whose Baseline Compiler is it Anyway? In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 207–220. IEEE (2024). https://doi.org/10.1109/cgo57630.2024.10444855

[24] Han, R., Zhao, J., Kim, H.: Enabling Fine-Grained Incremental Builds by Making Compiler Stateful. In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 221–232. IEEE (2024). https://doi.org/10.1109/cgo57630.2024.10444865

[25] Fehr, M., Fan, Y., Pompougnac, H., Regehr, J., Grosser, T.: First-Class Verification Dialects for MLIR. Proceedings of the ACM on Programming Languages **9**(PLDI), 1466–1490 (2025). https://doi.org/10.1145/3729309

[26] Xu, Z., Xu, H., Tian, Y., Zhou, X., Sun, C.: LPO: Discovering Missed Peephole Optimizations with Large Language Models. In: Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 1136–1150. ACM (2026). https://doi.org/10.1145/3779212.3790184

[27] Popescu, L., Lopes, N.P.: Exploiting Undefined Behavior in C/C++ Programs for Optimization: A Study on the Performance Impact. Proceedings of the ACM on Programming Languages **9**(PLDI), 348–371 (2025). https://doi.org/10.1145/3729260

[28] Mao, J., Ding, H., Zhai, J., Ma, S.: Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, pp. 639–653. ACM (2024). https://doi.org/10.1145/3620666.3651387

[29] Fan, Y., Regehr, J.: High-Throughput, Formal-Methods-Assisted Fuzzing for LLVM. In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 349–358. IEEE (2024). https://doi.org/10.1109/cgo57630.2024.10444854

[30] Kwon, J., Jang, B., Lee, J., Heo, K.: Optimization-Directed Compiler Fuzzing for Continuous Translation Validation. Proceedings of the ACM on Programming Languages **9**(PLDI), 627–650 (2025). https://doi.org/10.1145/3729275

[31] Lee, J., Kim, Y., Song, Y., Hur, C.-K., Das, S., Majnemer, D., Regehr, J., Lopes, N.P.: Taming undefined behavior in LLVM. In: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 633–647. ACM (2017). https://doi.org/10.1145/3062341.3062343

[32] Lobo, P., McIver, J., Mitenkov, G., Lee, J., Sundararajah, K., Lopes, N.P.: Towards Removing Undef Values from LLVM IR. Proceedings of the ACM on Programming Languages **10**(PLDI), 76–99 (2026). https://doi.org/10.1145/

3808250

[33] Yu, P., Wu, N., Donaldson, A.F.: Ratte: Fuzzing for Miscompilations in Multi-Level Compilers Using Composable Semantics. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 966–981. ACM (2025). https://doi.org/10.1145/3676641.3716270

[34] Pouchet, L.-N., Yuki, T.: PolyBench/C: the polyhedral benchmark suite, version 4.2.1 (beta). Benchmark suite documentation, The Ohio State University (2016). Available at https://sourceforge.net/projects/polybench/

[35] Guthaus, M.R., Ringenberg, J.S., Ernst, D., Austin, T.M., Mudge, T., Brown, R.B.: MiBench: A free, commercially representative embedded benchmark suite. In: Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538), pp. 3–14. IEEE (2001). https://doi.org/10.1109/wwc.2001.990739

[36] Callahan, D., Dongarra, J., Levine, D.: Vectorizing compilers: a test suite and results. In: Proceedings of the 1988 ACM/IEEE Conference on Supercomputing, pp. 98–105. IEEE Computer Society (1988)

[37] EEMBC: CoreMark: an EEMBC benchmark. Benchmark release, EEMBC (2009). Announced in June 2009. https://www.eembc.org/coremark/

[38] Maleki, S., Gao, Y., Garzarán, M.J., Wong, T., Padua, D.A.: An Evaluation of Vectorizing Compilers. In: 2011 International Conference on Parallel Architectures and Compilation Techniques, pp. 372–382. IEEE (2011). https://doi.org/10.1109/pact.2011.68

[39] de Moura, L., Bjørner, N.: Z3: An Efficient SMT Solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008). Lecture Notes in Computer Science, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24